

BinX TECH • AI & MACHINE LEARNING INTERNSHIP PROGRAM

WEEK 7

CNNs, RNNs & TRANSFORMERS — SPRINT 2

Convolutional, Recurrent & Attention-Based Architectures

The second sprint of the Phase 3 capstone: the specialized deep-learning architectures. Interns learn CNNs for images, RNNs/LSTMs for sequences, and the attention mechanism behind Transformers — then advance their project's core model and run the full Sprint 2 review cycle.

PHASE 3 — SPRINT 2 • 40 HOURS • 5 TRAINING DAYS • HANDS-ON

Prepared by BinX Tech
Palestine | Nablus

BinX
TECH

1. Week 7 Overview

Week	Week 7 of 10 — Phase 3: Deep Learning & Applied Project, Sprint 2
Total Hours	40 hours (full-time track) / 20 hours (part-time track, Weeks 13-14 combined)
Format	On-site / Remote / Hybrid — notebooks in Jupyter/Colab, committed to GitHub via feature branches
Focus	CNNs for images, RNNs/LSTMs for sequences, the attention mechanism and Transformers, pre-trained models; Sprint 2 core model development of the Phase 3 capstone project
Prerequisite	Weeks 1-6 completed; Sprint 1 reviewed and retrospective completed; baseline + first neural network in place
Mentor Supervision	Mentor Code & Notebook Review mid-sprint (Day 3) via pull request; Sprint Review + Retrospective on Day 5

2. Sprint 2 Structure

Week 7 is Sprint 2 of the Phase 3 capstone project, following the program's standard sprint cycle:

Event	Description
Sprint Planning (Day 1)	Define the Sprint 2 goal and backlog (core model development), incorporating the Sprint 1 retrospective improvement.
Daily Stand-up (Daily)	3-minute update: what was completed, what is next, any blockers.
Mentor Code & Notebook Review (Day 3)	Mentor reviews the notebook and code via GitHub pull request; structured comments on architecture and results.
Sprint Review (Day 5)	Demo the improved core model and its metrics. Incomplete tasks move to Sprint 3 with documented reasons.
Sprint Retrospective (Day 5)	What went well, what to improve, and one specific action for Sprint 3.

3. Week 7 Learning Objectives

- Explain why fully connected networks fail on images and how convolution solves it.
- Build a CNN with convolution, pooling, and dense layers, and apply transfer learning.
- Explain how RNNs and LSTMs process sequential data and why LSTMs handle long sequences better.
- Explain the attention mechanism and the core idea behind the Transformer architecture.
- Advance the Phase 3 project's core model using the architecture appropriate to its data type.
- Run the full Sprint 2 cycle: planning, stand-ups, mentor review, Sprint Review, and Retrospective.

4. Daily Schedule

Day	Hours	Topic Focus
Day 1	8 hrs	Sprint 2 planning; why CNNs — convolution, filters, feature maps
Day 2	8 hrs	Building a CNN: pooling, architecture; transfer learning with pre-trained models
Day 3	8 hrs	Sequential data: RNNs and LSTMs; Mentor Code & Notebook Review
Day 4	8 hrs	The attention mechanism and the Transformer architecture; pre-trained transformers
Day 5	8 hrs	Advancing the project's core model; Sprint Review & Retrospective

BinX
TECH

5. Day-by-Day Curriculum

Day 1 — Sprint 2 Planning & Convolutional Neural Networks — 8 hours

Learning Objectives

- Complete Sprint 2 planning and define the core-model backlog.
- Explain why dense networks fail on images.
- Explain convolution, filters, and feature maps, and why CNNs are efficient.

Key Topics

- Sprint 2 planning: goal, backlog, carrying forward the Sprint 1 retrospective
- Why dense networks fail on images
- Convolution: filters, feature maps, stride, padding
- Parameter sharing and translation invariance
- The feature hierarchy CNNs learn

Lesson Content

1.1 Sprint 2 Planning

Day 1 opens with Sprint Planning: define the Sprint 2 goal — advancing the project's core model — break it into backlog tasks, and carry forward the one improvement from the Sprint 1 retrospective. Sprint 2 is where the model that must beat the Week 6 baseline is genuinely developed, using the architecture that fits the project's data type (images → CNN, text/sequences → RNN/Transformer, tabular → the dense network from Week 6).

1.2 Why Dense Networks Fail on Images

A 200×200 color image has $200 \times 200 \times 3 = 120,000$ numbers. Feeding it into a dense network (Week 6) means the first layer alone needs millions of weights — it is computationally hopeless and ignores a key fact: in an image, nearby pixels are related, and a pattern (an edge, an eye) is the same wherever it appears. Dense networks throw away this spatial structure entirely.

1.3 Convolution: the Core Idea

A convolution slides a small filter (kernel) — a tiny grid of weights, e.g. 3×3 — across the image, computing a dot product at each position. Each filter learns to detect one specific pattern (a vertical edge, a curve, a texture). The output is a feature map showing where in the image that pattern appears. This is still the Week 2 dot product, applied locally and repeatedly across the image.

Concept	Meaning
Filter / kernel	A small grid of learnable weights that detects one pattern
Feature map	The output showing where the filter's pattern was found
Stride	How far the filter moves each step
Padding	Adding a border so the filter can process edge pixels

1.4 Why Convolution Wins

Convolution gives two decisive advantages. Parameter sharing: the same small filter is reused across the whole image, so a CNN needs vastly fewer weights than a dense network. Translation invariance: because the filter slides

everywhere, a pattern is detected no matter where it appears. Together these make CNNs both efficient and powerful on images.

A CNN learns a hierarchy: early layers detect simple patterns (edges), middle layers combine them (shapes, textures), and deep layers recognize whole objects (faces, cars). You do not design these features — the network learns them from data.

Hands-On Lab: Sprint 2 Kickoff & Convolution

- Step 1: Complete Sprint 2 planning and select the core-model backlog tasks.
- Step 2: On a sample image, apply a hand-defined edge-detection filter with a convolution and visualize the feature map.
- Step 3: Explain, in Markdown, why the same filter across the whole image needs far fewer weights than a dense layer.
- Step 4: Confirm whether the project's data type calls for a CNN, RNN/Transformer, or the dense network from Week 6, and record the decision.

Tools Used

TensorFlow/Keras • NumPy • Matplotlib • Jupyter/Colab • Git & GitHub

Day 2 — Building CNNs & Transfer Learning — 8 hours

Learning Objectives

- Build a full CNN with convolution, pooling, and dense layers.
- Apply data augmentation to reduce overfitting on image data.
- Use transfer learning with a pre-trained model to get strong results from little data.

Key Topics

- Pooling: shrinking feature maps
- Full CNN architecture: conv + pool + flatten + dense
- Data augmentation for computer vision
- Transfer learning with ResNet / EfficientNet / MobileNet
- Freezing and fine-tuning pre-trained layers

Lesson Content

2.1 Pooling

After convolution, a pooling layer shrinks the feature map by keeping only the strongest signal in each small region — max pooling takes the maximum value in each 2×2 window. Pooling reduces computation, controls overfitting, and makes the network robust to small shifts in the input. Convolution and pooling alternate to progressively distill the image into compact, meaningful features.

2.2 Full CNN Architecture

A typical CNN stacks convolution + pooling blocks to extract features, then flattens the result and feeds it into dense layers (Week 6) for the final classification:

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

model = Sequential([
    Conv2D(32, (3,3), activation="relu", input_shape=(128,128,3)),
    MaxPooling2D((2,2)),
```

```
Conv2D(64, (3,3), activation="relu"),
MaxPooling2D((2,2)),
Flatten(),
Dense(64, activation="relu"),
Dense(num_classes, activation="softmax"),
])
```

2.3 Data Augmentation

Image datasets are often too small to train a strong CNN without overfitting. Data augmentation artificially expands the dataset by applying random transformations — flips, rotations, zooms, shifts — so the network sees more varied examples and generalizes better. It is the standard first defense against overfitting in computer vision.

```
from tensorflow.keras.layers import RandomFlip, RandomRotation, RandomZoom
augment = Sequential([
    RandomFlip("horizontal"),
    RandomRotation(0.1),
    RandomZoom(0.1),
])
```

2.4 Transfer Learning

Training a CNN from scratch needs huge data and compute. Transfer learning reuses a model already trained on millions of images (ResNet, EfficientNet, MobileNet): you keep its learned feature-detecting layers and replace only the final classification layer with your own, then train on your smaller dataset. This is the single most practical technique in applied computer vision — strong results from little data.

```
from tensorflow.keras.applications import MobileNetV2
base = MobileNetV2(include_top=False, weights="imagenet",
                   input_shape=(128,128,3))
base.trainable = False # freeze the pre-trained feature extractor
model = Sequential([base, Flatten(),
                    Dense(num_classes, activation="softmax")])
```

If the Phase 3 project is the Image Classifier, transfer learning is almost always the right approach — it is exactly what the outline's project baseline expects for image tasks.

Hands-On Lab: Building and Transferring a CNN

- Step 1: Build and train a small CNN from scratch on an image dataset and record its accuracy.
- Step 2: Add data augmentation and compare the new validation curves to the previous run.
- Step 3: Apply transfer learning with a frozen pre-trained model (e.g. MobileNetV2) and compare accuracy and training time.
- Step 4: Document which approach performed best and why, in Markdown.

Tools Used

TensorFlow/Keras • Pre-trained models (Keras Applications) • Matplotlib • Jupyter/Colab (GPU)

Day 3 — RNNs & LSTMs for Sequential Data — 8 hours

Learning Objectives

- Explain why sequential data needs an order-aware architecture.
- Explain how an RNN's hidden state carries memory across a sequence.
- Explain the vanishing-gradient problem and how LSTMs solve it.

Key Topics

- Why order matters in sequential data
- RNNs: processing sequences with a hidden state
- The vanishing gradient problem
- LSTMs and GRUs: gated memory
- Embeddings for representing text

Lesson Content

3.1 Why Sequences Need a Different Architecture

CNNs handle spatial data (images); sequential data — text, time series, audio — has a different property: order matters, and each element depends on what came before. "The movie was not good" and "good, the movie was not" contain the same words but opposite meaning. A model for sequences must carry memory of previous elements.

3.2 Recurrent Neural Networks (RNNs)

An RNN processes a sequence one element at a time, maintaining a hidden state — a memory — that it updates at each step and passes forward. This lets it use context from earlier in the sequence when processing later elements. In principle an RNN can model any sequence; in practice, plain RNNs struggle to remember information across long sequences.

3.3 The Vanishing Gradient Problem

As gradients propagate back through many time steps during training, they shrink exponentially — the vanishing gradient problem — so plain RNNs effectively forget information from far back in the sequence. This is the specific weakness that LSTMs were designed to fix.

3.4 LSTMs and GRUs

An LSTM (Long Short-Term Memory) is an RNN with a gated memory cell: internal gates learn what to keep, what to forget, and what to output at each step. This lets it retain relevant information across long sequences and discard irrelevant information, solving the vanishing-gradient problem in practice. A GRU is a simpler, faster variant with fewer gates and similar performance.

```
from tensorflow.keras.layers import LSTM, Embedding, Dense
model = Sequential([
    Embedding(vocab_size, 128), # turn word indices into vectors
    LSTM(64),
    Dense(1, activation="sigmoid"), # e.g. sentiment: positive/negative
])
```

If the Phase 3 project is the Sentiment Analysis Tool, an LSTM (or a pre-trained transformer from Day 4) is the natural core model — text is sequential, so order-aware architecture matters.

Hands-On Lab: Sequence Modeling + Mentor Review

- Step 1: Build and train an LSTM on a sequential dataset (e.g. text sentiment or a time series) and record its metric.
- Step 2: Compare the LSTM against a plain RNN or a non-sequential baseline and note the difference.
- Step 3: Explain, in Markdown, why order-awareness helped on this data.
- Step 4: Open a pull request with the current project notebook for the mid-sprint Mentor Code & Notebook Review and address the feedback.

Tools Used

TensorFlow/Keras (LSTM) • Matplotlib • Git & GitHub (pull request)

Day 4 — Attention & Transformers — 8 hours

Learning Objectives

- Explain the limitation of RNNs that Transformers overcome.
- Explain the attention mechanism and why it is powerful and parallelizable.
- Use a pre-trained Transformer from Hugging Face for a text task.

Key Topics

- The limitation of step-by-step RNNs
- The attention mechanism: self-attention over all positions
- The Transformer architecture and positional encoding
- Pre-trained transformers: BERT, DistilBERT, GPT-2
- Hugging Face Transformers for fine-tuning and inference

Lesson Content

4.1 The Limitation of RNNs

Even LSTMs process sequences step by step, which makes them slow (no parallelism) and still imperfect at very long-range dependencies. The Transformer architecture, introduced in 2017, removed recurrence entirely and now underpins essentially all state-of-the-art NLP — including the models behind modern AI assistants.

4.2 The Attention Mechanism

Attention is the core idea. Instead of passing information step by step, attention lets every element in a sequence look directly at every other element and weigh how relevant each one is. When processing the word "it" in a sentence, attention lets the model look back and focus on the noun "it" refers to. This direct, weighted access to all positions is what makes Transformers so powerful — and it is parallelizable, so it trains far faster than an RNN.

Idea	What It Does
Self-attention	Each element weighs the relevance of every other element
Parallelism	All positions processed at once, not step by step — fast to train
Long-range context	Direct access to distant elements, no vanishing gradient

4.3 The Transformer Architecture

A Transformer stacks attention layers with feed-forward layers, plus positional encoding (since it has no inherent sense of order, position information is added explicitly to each element). Understanding it conceptually is enough for this program — you will use pre-trained Transformers rather than building one from scratch.

4.4 Pre-trained Transformers & Hugging Face

Just as transfer learning reuses pre-trained CNNs for vision, pre-trained Transformers (BERT, DistilBERT, GPT-2) are reused for NLP. The Hugging Face Transformers library makes loading and fine-tuning them straightforward, giving state-of-the-art text results with minimal code — the same efficiency principle as Day 2's transfer learning.

```
from transformers import pipeline
classifier = pipeline("sentiment-analysis") # loads a pre-trained model
result = classifier("This internship is excellent!")
# [{'label': 'POSITIVE', 'score': 0.9998}]
```

Hands-On Lab: Using a Pre-trained Transformer

- Step 1: Load a pre-trained transformer with the Hugging Face pipeline and run it on sample text.

- Step 2: If the project is text-based, apply the transformer to the project data and compare its metric to the Day 3 LSTM.
- Step 3: Explain, in Markdown, in one paragraph, how attention differs from an RNN's step-by-step memory.
- Step 4: Record which architecture (LSTM vs. transformer) will serve as the project's core model and why.

Tools Used

Hugging Face Transformers • TensorFlow / PyTorch • Jupyter/Colab (GPU)

Day 5 — Advancing the Core Model & Sprint Review — 8 hours

Learning Objectives

- Select the architecture that fits the project's data type.
- Advance and tune the project's core model to beat the baseline, logging experiments.
- Complete the full Sprint 2 review and retrospective cycle.

Key Topics

- Matching architecture to data type
- Advancing the core model over Sprint 1
- Logging experiments and comparing to the baseline
- Assembling Sprint Review evidence
- Sprint Review and Retrospective

Lesson Content

5.1 Choosing the Right Architecture for the Project

The week's central decision, now made concrete for the capstone:

Project Data Type	Core Architecture
Tabular (churn, house price, fraud)	Dense network (Week 6) or gradient boosting; deep learning optional
Images (image classifier)	CNN with transfer learning (Day 2)
Text (sentiment analysis)	LSTM or a pre-trained transformer (Days 3-4)
Sequences / time series	LSTM/GRU (Day 3)

Not every project needs a Transformer or CNN — a tabular churn or fraud project may be best served by the dense network or even a strong classical model. Matching the architecture to the data, rather than using the most complex option, is a mark of real understanding.

5.2 Advancing the Core Model

Sprint 2's deliverable is a meaningfully improved core model over Sprint 1: the right architecture, trained and tuned, with its score clearly beating the Week 6 baseline. Every experiment's configuration and metric should be logged (in MLflow or the notebook) and compared to the baseline — the reproducibility discipline required since Week 1 and by the program's acceptance criteria.

5.3 Preparing for Sprint Review

Assemble the evidence: the architecture chosen and why, training curves, and a metric table comparing the Sprint 2 model against both the baseline and the Sprint 1 network. Per the acceptance criteria: the notebook runs cleanly,

code is on the correct branch, the pull request is approved, results are documented in Markdown, and metrics are logged and compared.

5.4 Sprint Review & Retrospective

Sprint Review: demo the improved core model and its metrics to the mentor; unfinished tasks move to Sprint 3 with documented reasons. Retrospective: what went well, what to improve, and one concrete change for Sprint 3 (which focuses on NLP/CV integration and evaluation).

Hands-On Lab: Sprint 2 Close-Out

- Step 1: Confirm and justify the core architecture for the project based on its data type.
- Step 2: Train and tune the improved core model, logging each experiment's config and metric.
- Step 3: Assemble a metric table comparing the Sprint 2 model to the Week 6 baseline and the Sprint 1 network.
- Step 4: Ensure all Sprint 2 work is committed, the pull request is merged after mentor approval, and the notebook is documented.
- Step 5: Present the Sprint Review, then write the Retrospective with one concrete change for Sprint 3.

Tools Used

TensorFlow/Keras or Hugging Face • MLflow (experiment logging) • Matplotlib • Git & GitHub

6. Week 7 Deliverables

By the end of Week 7 (Sprint 2), every intern must submit the following to their mentor and GitHub repository:

- The Sprint 2 plan and a convolution demo showing a filter producing a feature map.
- A CNN notebook comparing a from-scratch CNN, augmentation, and transfer learning on image data.
- An LSTM notebook on sequential data compared against a non-sequential baseline.
- A pre-trained transformer applied to a text task via Hugging Face, with a short attention-vs-RNN explanation.
- The advanced project core model with a metric table beating the Week 6 baseline, plus logged experiments.
- All Sprint 2 work committed to GitHub via reviewed pull requests, plus the written Sprint Retrospective.

7. Week 7 Evaluation Criteria

Scored by the assigned mentor at the Sprint 2 review using the criteria below (extracted from the program's 100-point internal rubric).

Criterion	50-69 Developing	70-84 Proficient	85-100 Excellent
Understanding of architectures	Names architectures, unclear on when to use each	Explains CNNs, RNNs/LSTMs, and attention clearly	Deep understanding; matches architecture to data with justification
Deep learning proficiency	Builds models with heavy guidance	Builds CNNs/LSTMs and uses pre-trained models independently	Fluent; applies transfer learning and tuning effectively
Core model quality vs. baseline	Model built but weak baseline comparison	Improved model clearly documented vs. baseline	Rigorous; tuned core model decisively beats baseline
Sprint discipline	Misses stand-ups or planning steps	Completes full sprint cycle: plan, stand-ups, review, retro	Proactive sprint management; flags blockers early
Notebook & Git workflow	Sporadic commits, weak narrative	Regular commits, PR review, clear Markdown	Consistent, descriptive, reproducible, well-organized
Attendance & punctuality	3-6 absences	1-2 absences, on time	Perfect attendance, proactive

8. Technical Stack — Week 7

Computer Vision	TensorFlow/Keras (Conv2D, MaxPooling2D), Keras Applications (ResNet, EfficientNet, MobileNet)
Sequence Models	TensorFlow/Keras (LSTM, GRU, Embedding)
Transformers	Hugging Face Transformers (BERT, DistilBERT, GPT-2)
Experiment Tracking	MLflow
Compute & Workflow	Google Colab (GPU), Matplotlib, Git & GitHub (feature branches + pull requests)

9. Notes & Best Practices

Each architecture this week is the right tool for one data shape: CNNs exploit spatial structure, RNNs/LSTMs exploit order, Transformers use attention for direct long-range context. The professional skill is matching the architecture to the data — not reaching for the most complex model. Whatever the choice, it only counts if it beats the Week 6 baseline, logged and documented.